

In the name of God

Research on
methods to analyze test run results

Researcher
Mohadese Akhoondi

Master
Dr Mohammad Reza Sharbaf

University Of Isfahan

comprehensive list of methods, categorized by their focus, to diagnose and analyze unit test suite failures in software development. These methods combine debugging, data analysis, code inspection, and process evaluation to pinpoint issues effectively.

1. Log and Output Analysis : Examine test logs and outputs to identify failure details

Review error messages or stack traces in test logs for specific clues (e.g., `NullPointerException`, `AssertionError`). A stack trace pointing to a specific line in the code indicates where the failure occurred.

Use tools like `grep`, `Splunk`, or `ELK Stack` to filter and analyze test logs for patterns or recurring errors. For example Identifying repeated "timeout" errors in test runs.

Check test reports generated by frameworks (e.g., `JUnit`, `pytest`, `TestNG`) for failed test cases, including assertions that didn't hold. For example A `JUnit` report showing which assertions failed in a test case.

Enable detailed logging in the test suite or application to capture more context (e.g., variable states, inputs). For example Adding debug logs to trace variable values before a failure.

2. Code Inspection : Directly investigate the codebase and test execution.

Run the failing test in isolation to rule out interference from other tests (e.g., shared state issues).

3. Test Environment Analysis

Verify that the test environment (e.g., OS, dependencies, configurations) matches the expected setup.

Check versions of dependencies, frameworks, or external services (e.g., databases, APIs) for compatibility issues.

Investigate if system resources (e.g., CPU, memory, disk) were overloaded during the test run.

Review configuration files or environment variables for incorrect settings. For example a test failing because a database connection string is misconfigured.

4. Test Design and Logic Analysis : Evaluate the test suite's design and assumptions.

Check if assertions are correct and test the intended behavior.

Verify that input data or mocks used in tests are valid and representative.

Ensure tests are independent and not affected by shared state or test order.

Check if tests cover edge cases or boundary conditions that might cause failures. For example A test failing because it didn't account for a null input.

Identify tests that fail intermittently due to non-deterministic behavior (e.g., race conditions, random inputs). For example Rerunning a failing test multiple times to confirm flakiness.

5. Comparative Analysis : Compare test runs or code versions to isolate issues.

Compare current test results with previous successful runs to identify changes that caused failures.

Run tests with different configurations or versions of the code to isolate the cause.

Correlate test failures with recent code changes using version control history (Use git blame or git bisect to identify commits introducing failures.).

6. Root Cause Analysis (RCA) Techniques :

Repeatedly ask "why" to drill down to the root cause of a failure. For example Why did the test fail? → Assertion error. Why? → Incorrect output. Why? → Bug in the logic.

Categorize potential causes (e.g., code, environment, test design) to identify the root issue.

Fault Tree Analysis which is Modeling logical relationships between events leading to test failure.

Identify untested or poorly covered code paths that may contribute to failures (Test Coverage Gap).

a list of GitHub projects that provide analytical reports on test execution results. These tools help developers better analyze test outcomes

Test Reporting Tools

1. **Test Reporter:** A tool for displaying test results in the GitHub UI, compatible with frameworks like xUnit and JUnit.
2. **GitHub Test Reporter:** Enables publishing and viewing test reports directly in GitHub Actions.
3. **XCTestHTMLReport:** Generates Xcode-style HTML reports for Unit and UI tests.
4. **CI::Reporter:** A Ruby framework plugin for generating XML reports for CI systems.
5. **Allure Report:** A multilingual reporting tool with clear graphical dashboards, supporting Selenium, TestNG, and more.
6. **Cluecumber:** A Maven plugin for generating Cucumber BDD test reports.
7. **TestBeats:** A tool for publishing test results to platforms like Microsoft Teams and Slack.
8. **ExtentReports:** A reporting API for automated tests, particularly for Java.
9. **ReportPortal:** An advanced analytics reporting service with Jenkins integration.
10. **ReportNG:** An HTML/XML reporting plugin for TestNG (no longer maintained).
11. **pytest-html:** A plugin for generating HTML reports for pytest results.
12. **jest-html-reporter:** Generates HTML reports for Jest test results.
13. **Spock Reports:** Produces HTML reports for Spock specifications.

Project Name	Description	GitHub Link
Test Reporter	Displays test results in the GitHub UI for frameworks like xUnit and JUnit.	https://github.com/dorny/test-reporter
GitHub Test Reporter	Publish and view test reports in GitHub Actions; suitable for all	https://github.com/ctrf-io/github-test-reporter

	major frameworks.	
XCTestHTMLReport	Generates Xcode-like HTML reports for Unit and UI tests, with support for parallel testing.	https://github.com/TitouanVanBelle/XCTestHTMLReport
CI::Reporter	Plugin for Ruby frameworks like Test::Unit; produces XML reports for CI.	https://github.com/ci-reporter/ci_reporter
Allure Report	Multilanguage reporting tool with graphical charts; works with Selenium, TestNG, and more.	https://github.com/allure-framework/allure2
Cluecumber	Maven plugin that builds test reports for Cucumber BDD from JSON files.	https://github.com/trivago/cluecumber
TestBeats	Publishes test results to platforms like Microsoft Teams and Slack; supports multiple frameworks.	https://github.com/test-results-reporter/testbeats
ExtentReports	Reporting API for automated	https://github.com/extent-framework/extentreports-java

	tests—especially Java—with extensive customization options.	
ReportPortal	Hosted reporting service with advanced analytics; integrates with Jenkins, Jira, and Cucumber.	https://github.com/reportportal/reportportal
ReportNG	HTML/XML reporting plugin for TestNG (no longer maintained).	https://github.com/dwdyer/reportng
pytest-html	Plugin that creates HTML reports for pytest results; ideal for Python tests.	https://github.com/pytest-dev/pytest-html
jest-html-reporter	Generates an HTML report for Jest test results; suited to JavaScript projects.	https://github.com/Hargne/jest-html-reporter
Spock Reports	Produces HTML reports for Spock specifications; intended for Groovy tests.	https://github.com/renatoathaydes/spock-reports

Resources

[Allure Report](#)

[TestBeats](#)

[pytest-html](#)

Output parameters of existing project on github

1.

<https://github.com/marketplace/actions/test-reporter>

Name	Description
conclusion	success or failure
passed	Count of passed tests
failed	Count of failed tests
skipped	Count of skipped tests
time	Test execution time [ms]
url	Check run URL
url_html	Check run URL HTML

2.

<https://github.com/ctrif-io/github-test-reporter?tab=readme-ov-file>

Test Summary

Tests	Passed	Failed	Skipped	Pending	Other	Flaky	Duration
4	2	1	1	0	0	1	00:00:20

Failed Test Summary

Name	Status	Failure Message
should display profile information	failed	Assertion Failure: profile mismatch

Flaky Test Summary

Name	Status	Retries	Flaky
should be able to update profile	passed	2	Yes

Detailed Test Results

Name	Status	ms	Flaky
should be able to login	passed	1200	
should display profile information	failed	800	
should be able to update profile	passed	1200	Yes
should be able to logout	skipped	0	

github-actionsbotcommented 5 minutes ago

Test Results

2 failed

5 passed

3 flaky

Details

10 tests

11.0s

jest

My App v1.0.0

build-and-test → github-report-test #118

Feat/GitHub style report #125

▼ Failed Tests

login.test.ts → profile → should display title

network.test.ts → network → should fail to update profile on network failure

▼ Flaky Tests

login.test.ts → profile → should be able to update profile

Insights

Average Tests per Run	Total Flaky Tests	Total Failed	Slowest Test (p95)
17	1	1	19ms

Failed Rate

Overall Fail Rate: 1.23%

Test	Runs	Pass	Fail	Fail Rate %
stripAnsi returns the same string if no ANSI codes are present	10	0	1	10.00

Flaky Rate

Overall Flaky Rate: 3.55%

Test	Attempts	Pass	Fail	Flaky Rate %
createStatusCheck should create a successful check run when no tests failed	13	10	3	23.07

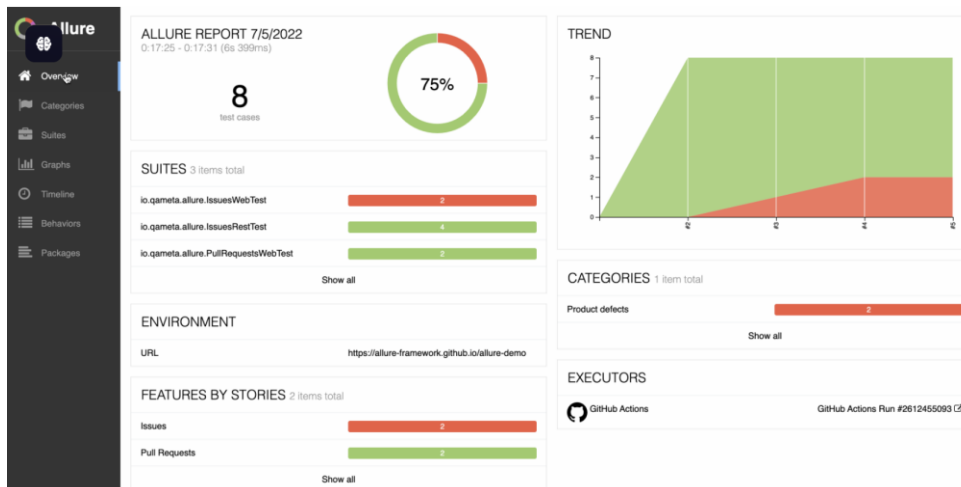
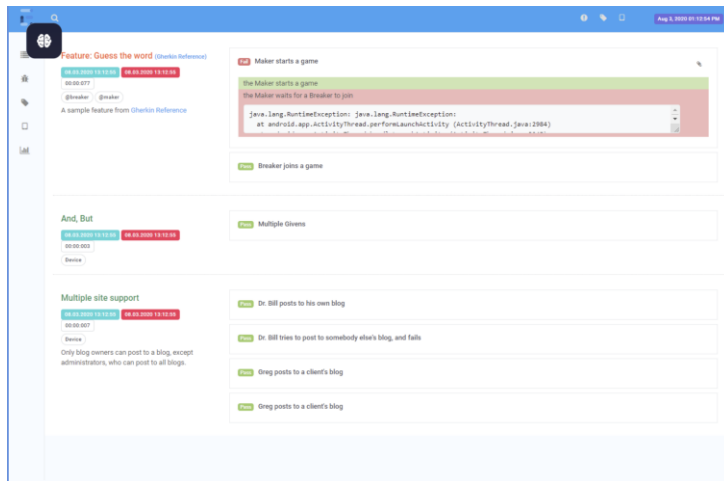
Slowest Tests

Test	Runs	Fail	Fail Rate %	Average Duration (p95)
stripAnsi throws a TypeError if the input is not a string	10	0	0%	19ms
getEmoji returns the correct emoji for "passed"	10	0	0%	5ms
createStatusCheck should create a successful check run when no tests failed	10	0	0%	5ms
getEmoji returns the correct emoji for "pending"	10	0	0%	1ms
getEmoji returns the correct emoji for "build"	10	0	0%	1ms
stripAnsi returns the same string if no ANSI codes are present	10	0	0%	1ms
createStatusCheck should create a failed check run when tests failed	10	0	0%	1ms
createStatusCheck should truncate summary if it exceeds 65000 characters	10	0	0%	1ms

3.

This site includes all github links of Open Source Test Reporting Tools

<https://www.softwaretestingmagazine.com/tools/open-source-test-reporting-tools/>



Allure Report

Allure

Overview

Categories

Suites

Graphs

Timeline

Behaviors

Package

Categories

order: name duration status

Marks: [icon] [icon] [icon] [icon]

Product defects

Element not found (By.xpath: //div[@class='something']) Expected: visible or transparent: visible or have css value opacity=0 Screenshot: file:/Users/eroshenkoam/Developer/eroshenkoam/webdriver-coverage-example/build/reports/tests/1603973516437.0.png Page source: file:/Users/eroshenkoam/Developer/eroshenkoam/webdriver-coverage-example/build/reports/tests/1603973516437.0.html Timeout: 4 s

#1 Closing new issue for authorized user 1s 006ms

Element should text 'Some issue title here' (By.xpath: //a[@href='/eroshenkoam/alture-example/']) Element: 'a class='v-align-middle'>another text Screenshot: file:/Users/eroshenkoam/Developer/eroshenkoam/webdriver-coverage-example/build/reports/tests/1603973703632.0.png Page source: file:/Users/eroshenkoam/Developer/eroshenkoam/webdriver-coverage-example/build/reports/tests/1603973703632.0.html Timeout: 4 s

io.qameta.allure.IssuesWebTest.shouldCloseIssue

Failed Closing new issue for authorized user

Overview History Retries

Element not found (By.xpath: //div[@class='something']) Expected: visible or transparent: visible or have css value opacity=0 Screenshot: file:/Users/eroshenkoam/Developer/eroshenkoam/webdriver-coverage-example/build/reports/tests/1603973516437.0.png Page source: file:/Users/eroshenkoam/Developer/eroshenkoam/webdriver-coverage-example/build/reports/tests/1603973516437.0.html Timeout: 4 s

Tags: web, bugpass

Categories: Product defects

Severity: normal

Duration: 1s 006ms

Owner: eroshenkoam

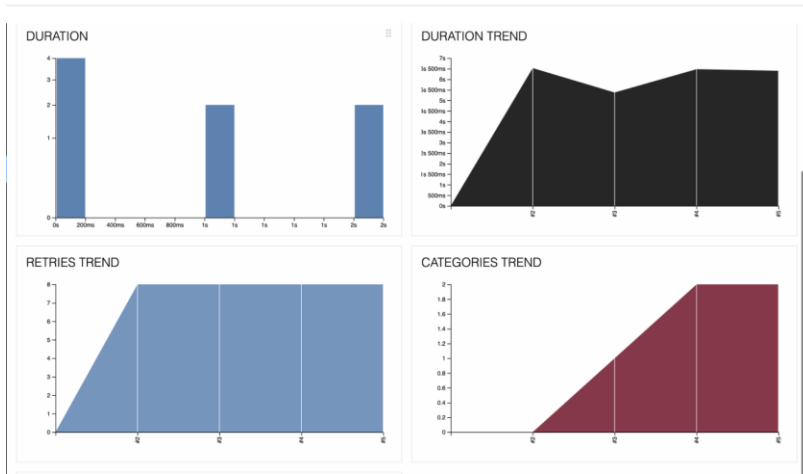
Execution

Set up

Test body

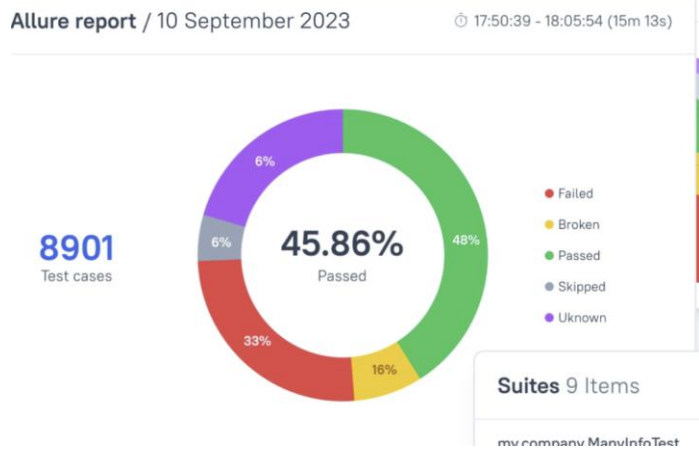
Open issues page 'alture-framework/alture2' 2 parameters, 1 attachment 1s 006ms

Tear down



RETRIES TREND

CATEGORIES TREND



Also more tools

Tool Name	GitHub Link	Output Parameters
Allure Framework	https://github.com/allure-framework	Detailed test-case steps; attachments (screenshots, logs); defect classifications; historical data; multi-OS compatibility
ExtentReports / Klov	https://github.com/extent-framework	Web-based reports; historical analysis; interactive charts; detailed test logs; integration with testing frameworks
ReportPortal	https://github.com/reportportal/reportportal	Real-time reports; analytics; full test-history logs; AI/ML analysis; defect reports; CI/CD integration
CI::Reporter	https://github.com/ci-reporter/ci-reporter	XML reports in JUnit format: test suite & case details, statuses, execution times, failure messages
Cluecumber Report Maven Plugin	https://github.com/trivago/cluecumber-report-plugin	Aggregated reports from Cucumber JSON: feature, scenario, and step details;

		pass/fail statuses
Go-allure	https://github.com/robotomize/go-allure	Allure reports for Go tests: detailed test information and attachments
HtmlTestRunner	https://github.com/oldani/HtmlTestRunner	HTML reports for Ruby tests: test-case names, statuses, and error messages
Jasmine Reporters	https://github.com/larrymyers/jasmine-reporters	JUnitXmlReporter for Jasmine: XML reports with test details, statuses, and times
jest-html-reporter	https://github.com/Hargne/jest-html-reporter	HTML summaries for Jest: test-suite and test-case details; pass/fail statuses
jest-html-reporters	https://github.com/Hazyzh/jest-html-reporters	HTML reports for Jest with detailed suite/case results and statuses
karma-htmlfile-reporter	https://github.com/matthias-schuetz/karma-htmlfile-reporter	Styled HTML files for Karma: test results per browser; separate tables for each browser
Next Gen Report	https://github.com/iambalabharathi/nxgreport	E-mailable HTML/PDF reports for Ruby tests: test results and summaries

ReportBuilder	https://github.com/rajatthareja/ReportBuilder	Mobile-friendly HTML/JSON reports for Cucumber: test details and retry files
ReportNG	https://testng.org/reportng/	HTML and XML reports for TestNG: categorized test results with detailed information
Spock Reports	https://github.com/renatoathaydes/spock-reports	HTML reports for Spock: summary index, customizable templates, detailed test info
XCTestHTMLReport	https://github.com/TitouanVanBelle/XCTestHTMLReport	Xcode-like HTML reports: Unit/UI test results, parallel-testing support, attachments

Here's an AI Test Reporter for Playwright — Summarize Test Results

<https://medium.com/@ma11hewthomas/playwright-with-ai-generated-test-summaries-fd669c9ce458>

Here is an example of the stdout:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Reporter Summary

❌ Failed Test: should handle login with invalid credentials
The test failed because the server responded with an unexpected status code. Instead of receiving the anticipated 401 status code for invalid credentials, the server returned a 500 internal server error. This suggests there might be a problem on the server-side rather than with the authentication logic itself. To resolve this, you should first check the server logs for any errors that occurred during the login attempt. It's possible there's an unhandled exception or a database connectivity issue causing the server to crash instead of properly handling the invalid login. You might also want to review the server-side authentication code to ensure it's correctly catching and handling authentication failures. Additionally, temporarily adding more detailed error logging on the server could help pinpoint the exact cause of the 500 error during the login process.

❌ Failed Test: should process payment with valid card details
The test failed because the payment gateway timed out after 30 seconds without responding. This issue could be caused by network latency, high server load, or temporary service disruption at the payment gateway. To resolve this, you might want to check the payment gateway's status page for any reported outages. If there are no known issues, try increasing the timeout threshold for the test to account for potential slow responses. It's also worth investigating your network connection to ensure it's stable. If the problem persists, you may need to contact the payment gateway's support team to see if there are any issues on their end. In the meantime, you could consider implementing a retry mechanism in your payment processing logic to handle occasional timeouts more gracefully.

❌ Failed Test: should retrieve user orders
The test failed because of a network error when trying to fetch data from the orders API. This issue likely occurred in the OrderTest.js file, specifically on line 23. The problem could be due to several factors, such as the API being temporarily unavailable, network connectivity issues, or incorrect API endpoint configuration. To resolve this, you might want to check if the orders API is up and running, verify your network connection, and ensure that the API endpoint URL is correct in your test setup. It's also worth confirming that any necessary authentication or headers are properly set for the API request. If the issue persists, you could try adding a timeout or a retry mechanism to handle temporary network glitches.

Overall Summary:
This test run has 3 failed tests, revealing potential issues across different system components. The failures suggest problems with server stability, external service reliability, and network connectivity.
The login test failure points to a server-side issue, as it's returning a 500 error instead of the expected 401 for invalid credentials. This indicates a possible unhandled exception in the authentication process.
The payment processing test timeout suggests either an external dependency problem with the payment gateway or network latency issues. This could impact the system's ability to handle financial transactions reliably.
The user orders retrieval failure due to a network error highlights potential API or connectivity problems. This could affect the system's ability to access and display critical user data.
These failures collectively indicate that the system's robustness in handling external services, network issues, and server-side errors may need improvement. Focusing on error handling, timeout management, and server stability could address the root causes behind these test failures.
Updated report saved to c:\rf-reports\c\rf-report.json
- ai-test-reporter-c\rf-report.json
```

AI Summary

Failed Test ❌

should display title

AI Summary ✨

The test failed because the page title didn't match the expected value within the given timeout period. To resolve this issue, you should first check if the title of the page is correct in your application. It seems there might be a typo or a misunderstanding about what the actual title should be. If 'Common Test Report Format' is indeed the correct title, you'll need to update your test expectations. On the other hand, if 'Uncommon Test Report Format' is the intended title, you'll need to fix the title in your application code.

Another possibility is that the page might be taking longer to load than expected, causing the title to not appear within the 5-second timeout. In this case, you could try increasing the timeout duration in your test to give the page more time to load completely.

<https://www.testmo.com/github-test-management/>

The screenshot displays the Testmo web application interface. On the left, there's a sidebar with navigation icons. The main area is divided into two panels. The left panel shows a 'Dashboards' section with a list of tests and their status (e.g., 'Test', 'Status', 'Failed', 'Passed'). The right panel shows a list of issues, with columns for 'Author', 'Label', 'Projects', 'Milestones', 'Assignee', and 'Sort'. The issues are listed with their IDs (e.g., #18, #15, #14, #13, #12, #11, #10, #9, #8) and their status (e.g., 'Open', 'Closed'). The interface is clean and modern, with a dark theme.

<https://reportportal.io/>

I used Demo of this product and it's all output parameters

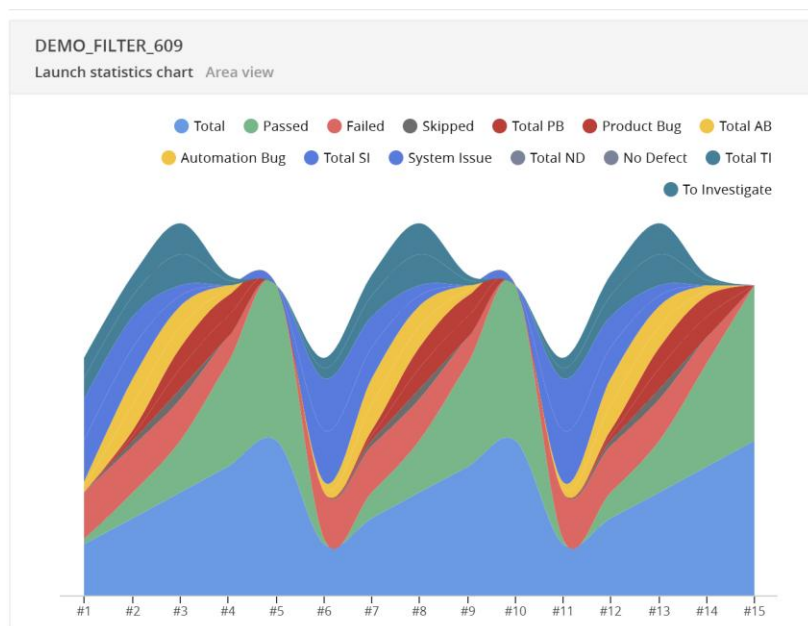


Table of above picture parameters

Legend Entry	Meaning (per launch)	Typical Use-Case for a Developer
Total (blue)	Total number of test items executed in that launch (passed + failed + skipped).	Quick sense of launch size / workload.
Passed (green)	Items whose assertions all succeeded.	Confirms coverage that is already stable.
Failed (red)	Items that ended with a FAILED status but have not yet been assigned any defect type.	Pool of failures that still need triage.
Skipped	Items the runner declared skipped / ignored / disabled.	Reveals tests muted by feature flags, @Disabled, etc.
Product Bug	A failed item tagged as a defect in the application under test.	Signals real functional issues to product engineers.
Total PB	Sum of all Product-Bug sub-types (e.g., PB001-UI, PB002-API).	High-level view of product quality.
Automation bug	Failure caused by an error in the test script, framework, or data.	Points QA/automation engineers to flaky or outdated tests.

Total AB	Aggregate of every Automation-Bug subtype.	Shows how much of the red is self-inflicted by the test harness.
System Issue	Failure rooted in the infrastructure (CI node down, DB unreachable, OOM, etc.).	Escalate to DevOps rather than product devs.
Total SI	Sum of all System-Issue subtypes.	Separates infra instability from code regressions.
No Defect	Item marked as “works as designed” or “expected change”; not counted as a bug.	Keeps noise out of bug metrics.
Total ND	Combined view of all “No Defect” sub-types.	High-level false-positive gauge.
To Investigate	Default tag for a new failure that no one has triaged yet.	Personal inbox for the next triage round.
Total TI	Total of every “To Investigate” subtype (if any).	Lets leads see the triage backlog size.

Demo Api Tests_432 Most failed test-cases table (TOP-50)			
LAUNCH NAME: Demo Api Tests #15 BASED ON: Failed			
TEST CASE	ISSUES IN EXECUTI...	% OF ISSUES	LAST ISSUE
History table. Extended functionality. Perm...	 12 of 15	80.00%	2 hours ago
first test	 12 of 15	80.00%	2 hours ago
beforeClass	 6 of 12	50.00%	2 hours ago
testLaunchStatusAfterDeletingTest	 6 of 12	50.00%	2 hours ago
afterClass	 6 of 15	40.00%	2 hours ago
testFilterPositive	 6 of 15	40.00%	2 hours ago
testFilterNegative	 6 of 15	40.00%	2 hours ago